



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C11

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.

Szablony klas

Szablony klas (ang. generics) to narzędzie w programowaniu obiektowym, które pozwala na tworzenie klas, metod i interfejsów z parametrami typów. Dzięki szablonom możemy pisać bardziej elastyczny i wielokrotnego użytku kod, który może działać z różnymi typami danych bez konieczności duplikowania kodu dla każdego typu. Szablony pozwalają na tworzenie klas i metod, które mogą działać z dowolnym typem danych. Dzięki szablonom możemy uniknąć błędów związanych z rzutowaniem typów. Ponadto szablony pozwalają na pisanie bardziej ogólnego kodu, który może być używany w różnych kontekstach. Poniżej znajduje się przykład szablonu klasy w C#, który obsługuje typy liczbowe. Klasa MathOperations zawiera metody do dodawania, odejmowania, mnożenia i dzielenia liczb.

Przykład 1. Szablony klas

```
using System;
namespace SzablonyKlas
{
    // Szablon klasy MathOperations dla typów liczbowych
    public class MathOperations<T> where T : struct, IComparable, IFormattable, IConvertible,
    IEquatable<T>
    {
        // Metoda dodawania
        public T Dodaj(T a, T b)
        {
            dynamic x = a;
            dynamic y = b;
            return x + y;
        }

        // Metoda odejmowania
        public T Odejmij(T a, T b)
        {
            dynamic x = a;
            dynamic y = b;
            return x - y;
        }
    }
}
```

```

    }

    // Metoda mnożenia
    public T Pomnóż(T a, T b)
    {
        dynamic x = a;
        dynamic y = b;
        return x * y;
    }

    // Metoda dzielenia
    public T Podziel(T a, T b)
    {
        dynamic x = a;
        dynamic y = b;
        if (y == 0)
        {
            throw new DivideByZeroException("Nie można dzielić przez zero.");
        }
        return x / y;
    }

    // Metoda obliczania reszty z dzielenia
    public T Modulo(T a, T b)
    {
        dynamic x = a;
        dynamic y = b;
        return x % y;
    }

    // Metoda potęgowania z obsługą wyjątków
    public T Power(T a, int b)
    {
        try
        {
            dynamic x = a;
            return (T)Math.Pow(x, b);
        }
        catch (OverflowException)
        {
            throw new OverflowException("Wynik potęgowania przekroczył zakres typu
liczbowego.");
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        // Przykład użycia klasy MathOperations z typem int
        MathOperations<int> operacjeInt = new MathOperations<int>();
        Console.WriteLine("Dodawanie int: " + operacjeInt.Dodaj(5, 3));
        Console.WriteLine("Odejmowanie int: " + operacjeInt.Odejmij(5, 3));
        Console.WriteLine("Mnożenie int: " + operacjeInt.Pomnóż(5, 3));
        Console.WriteLine("Dzielenie int: " + operacjeInt.Podziel(5, 3));
        Console.WriteLine("Modulo int: " + operacjeInt.Modulo(5, 3));
        Console.WriteLine("Potęgowanie int: " + operacjeInt.Power(5, 3));

        // Przykład użycia klasy MathOperations z typem double
        MathOperations<double> operacjeDouble = new MathOperations<double>();
        Console.WriteLine("Dodawanie double: " + operacjeDouble.Dodaj(5.5, 3.3));
        Console.WriteLine("Odejmowanie double: " + operacjeDouble.Odejmij(5.5, 3.3));
        Console.WriteLine("Mnożenie double: " + operacjeDouble.Pomnóż(5.5, 3.3));
        Console.WriteLine("Dzielenie double: " + operacjeDouble.Podziel(5.5, 3.3));
        Console.WriteLine("Modulo double: " + operacjeDouble.Modulo(5.5, 3.3));
        Console.WriteLine("Potęgowanie double: " + operacjeDouble.Power(5.5, 3));
    }
}

```

Deklaracja szablonu klasy

```
public class MathOperations<T> where T : struct, IComparable, IFormattable, IConvertible, IEquatable<T> -
```

Klasa MathOperations jest szablonem, który działa z typami liczbowymi (struct) i implementuje kilka interfejsów, aby zapewnić, że typy te mogą być porównywane, formatowane, konwertowane i porównywane pod względem równości.

Metody dodawania, odejmowania, mnożenia, dzielenia, obliczania reszty z dzielenia i potęgowania

Każda metoda używa dynamicznych typów (dynamic), aby umożliwić operacje arytmetyczne na typach liczbowych.

Obsługa wyjątków w metodzie potęgowania

Metoda Power obsługuje wyjątek OverflowException, który może wystąpić, gdy wynik potęgowania przekroczy zakres typu liczbowego.

Przykłady użycia

W metodzie Main tworzymy instancje klasy MathOperations dla typów int i double i wykonujemy operacje arytmetyczne.

Zadania do samodzielnego rozwiązania

Zadanie 1. Rozszerzenie klasy MathOperations

Dodaj do klasy MathOperations metody do obliczania reszty z dzielenia (Modulo) oraz potęgowania (Power).

Zadanie 2. Obsługa innych typów liczbowych

Zmodyfikuj klasę MathOperations, aby obsługiwała również typ decimal.

Zadanie 3. Walidacja danych wejściowych

Dodaj walidację danych wejściowych do metod klasy MathOperations, aby upewnić się, że przekazane wartości są poprawne (np. nie są null).